

Assigning Multiple Values To Variables

1) Assign to Multiple Variables

Example: `a, b = 1, 2`

2) Simplifies, Improves readability

Python allows assigning values to multiple variables in a single line, simplifying the process and making the code efficient, concise & of course, easy to read.

3) Simple & Efficient Swaps

4) Smoothy Solve Exchanges

Example:

```
a, b, c = 5, 10, 15
print(a, b, c)
```

4

Assigning Multiple Values to Variables

Example:

```
x, y = 5 + 3, 10 * 2
print("value of x is:", x, "value of y is:", y)
```

Please do the Practice these techniques to enhance your code, Skills.

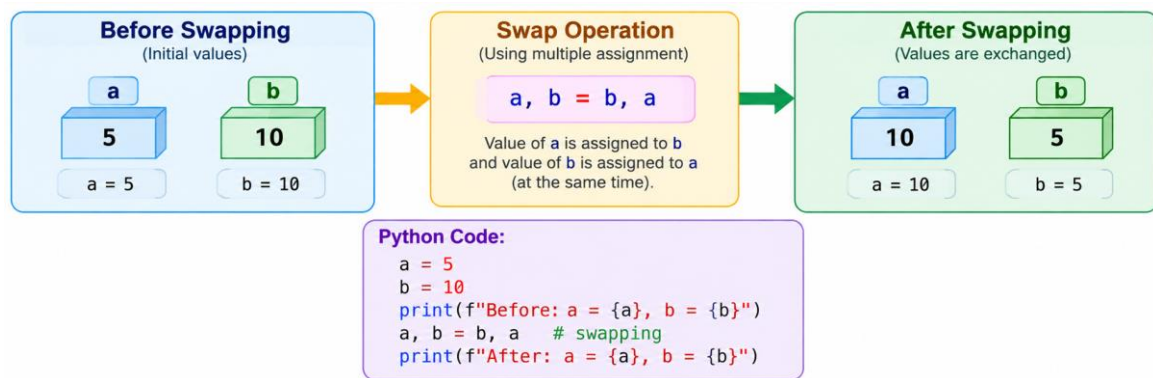
Traditional:

```
a = 5
b = 10
c = 15
print("value of a is:", a, "value of b is:", b, "value of c is:", c)
```

Swapping values

```
a = 5
b = 10
a, b = b, a
print("value of a is:", a)
print("value of b is:", b)
```

In Python, we can swap two variables without a temporary variable.



Important: The number of variables on the left should match the number of values on the right.

Note: Feature is simpler and more elegant compared to other programming language

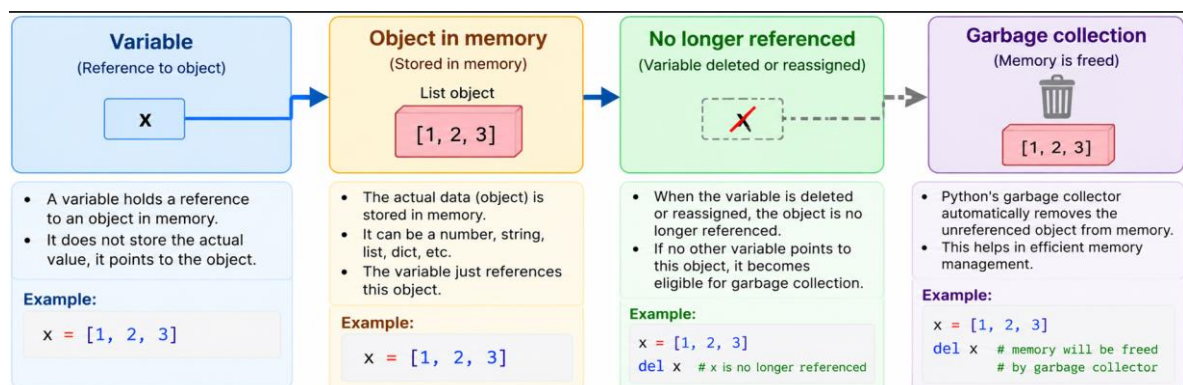
Memory Management by Variables

Python manages memory automatically.

When objects are no longer referenced, Python can **reclaim their memory through garbage collection**.

Python handles memory management behind the scenes, so you don't need to worry about cleaning space or tracking or keeping track, how much memory has you used?

If something is no longer being used, meaning nothing points to it, Python removes it to free up the space. This process is called **"Garbage Collection"** – gc.



```
data = "CloudNuts"
data = "Python"
# The old object may become eligible for garbage collection.
```

- The programmer normally does not manually free ordinary Python objects.
- Python's automatic memory management helps programs run without explicit deallocation in typical code.
- The scan uses a simplified view; actual Python memory management includes reference counting and cyclic garbage collection.

Displaying and Formatting Variable Output

In Python Displaying & Formatting variable output is an important skill that helps you “**Present Data Clearly**” and make your programs more user friendly.

Real Example:

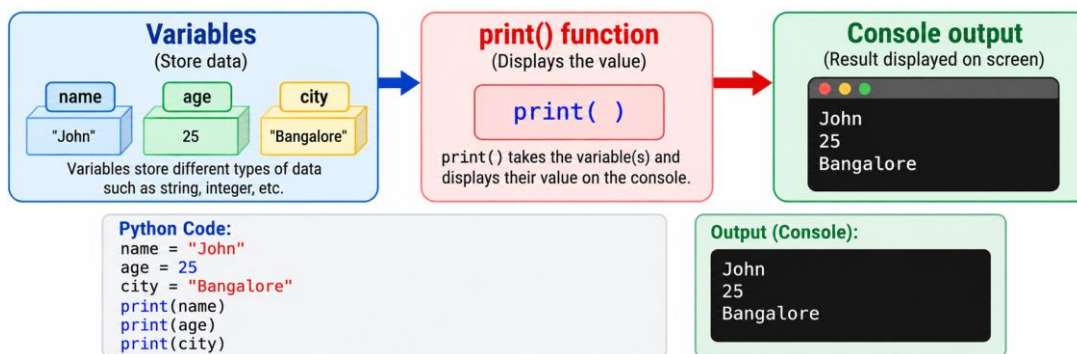
1. Banking System to Display Account Balance & Transaction Histories
2. E-Commerce Platforms, to show Product details Pricing & User Order summaries
3. In Gaming. Used to display Scores player start.

Print function which is used to display text or variable values On the Screen.

Example: `print("Hello, World")`

```
username = "CloudNuts"  
print(username)  
print("Hello,", username)
```

More Descriptive, `print("Hello,"+ username)`



f-Strings and format () – Formatted String Literals

It is easier & more readable

Simply, add and “f” before the opening quotes & place the variables inside curly braces within the string.

A. f-Strings

```
username = "CloudNuts"  
age = 4  
print(f"Hello, {username}")  
print(f"{username} is {age} years old")
```

An f-string places an f before the opening quote and allows expressions/variables inside curly braces.

B. format () – Format Function

This was widely used before f string became popular in python version 3.6

To use the format function, you write placeholders like curly braces, open curly braces closed inside a string and then use dot format function to fill them with variable values.

```
username = "CloudNuts"  
age = 4  
print("Hello, {}".format(username))  
print("{} is {} years old".format(username, age))
```

C. New lines and tabs

\n for New Line

\t for Add Tab

```
print("Username:\tAlice\nAge:\t25")
```

